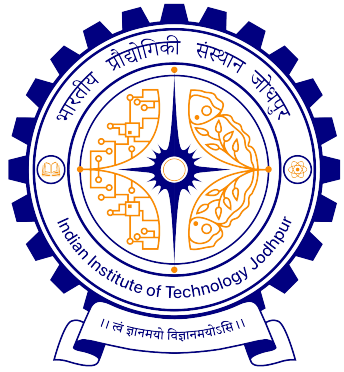
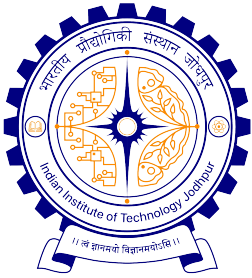




Introduction to Computer Science (CSL1010)

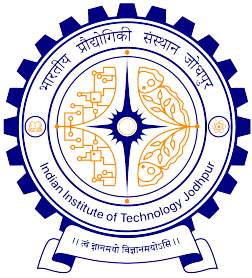
C Basics– Storage Class & Command Line Arguments L11

Suchetana Chakraborty
Department of Computer Science, IIT Jodhpur
Spring, 2026



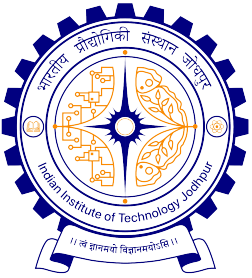
Storage Class

- A variable's storage class determines **when the cell is allocated** and **how long the cell remains in existence**.
- The storage class of a variable or a function influences its **visibility** - that is, where in a program the variable or function can be referenced.
- The commonly used storage classes are:
auto, extern, static.



Data Storage in Memory

- Variables may be *automatic* or *static*
- *Automatic variables* may *only* be declared *within* functions and compound statements (*blocks*)
 - Storage *allocated* when function or block is entered
 - Storage is *released* when function returns or block exits
- Parameters and result are (somewhat) like automatic variables
 - Storage is *allocated* and *initialized* by *caller* of function
 - Storage is *released* after function *returns* to caller.



Example

```
int fcn (float a, int b) {
```

```
    int i;
```

```
    double g;
```

```
    for (i = 0; i < b; i++) {
```

```
        double h = i*g;
```

```
        loop body – may access a, b, i, g, h  
    } // for(i...)
```

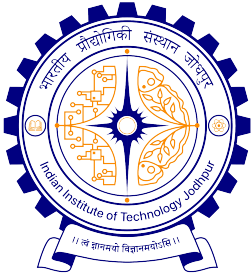
```
    fcn body – may access a, b, i, g
```

```
} // int fcn( ... )
```

i is
te

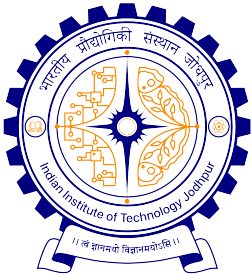
g is visible from this point
to end of **fcn**

h is only visible from this
point to end of loop!



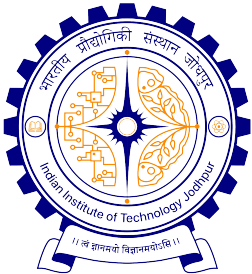
Idiosyncrasies

- In traditional *C* & *Visual Studio*
 - *All variables* must be declared at *beginning* of function or compound statement (i.e., before first *statement*); visible from that point on
- In **gcc**
 - *Variables* may be declared anywhere in function or compound statement; visible from that point on
- In *C99* & *C++*
 - *Loop variables* may be declared in **for** statement; visible only to end of loop body, but not beyond



Static Data – *Very* different

- *Static* variables may be declared within or outside of functions
 - Storage allocated when program is initialized
 - Storage is released when program exits
- *Static* variables outside of functions *may* be visible to linker
- Compiler sets aside storage for all static variables at compiler or link time
- Values *retained* across function calls
- Initialization *must* evaluate to compile-time constant



Static Variable Examples

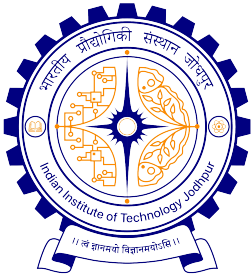
```
int j;           //static, visible to linker & all functs
static float f;  // not visible to linker, visible to
                // to all functions in this program
```

```
int fcn (float a, int b) {
    // nothing inside of {} is visible to linker
```

```
    int i = b;      //automatic
    double g;       //automatic
    static double h; //static, not visible to
                    // linker; value retained from call to call
```

body – may access j, f, a, b, i, g, h

```
} //    int fcn( ... )
```



Static Variable Examples (continued)

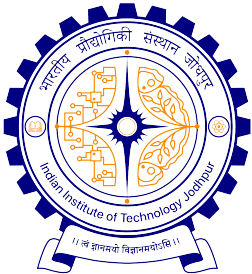
```
int j;      //static, visible to linker & all functs
static float f; // not visible to linker, visible to
              // to all functions in this program
```

```
int fcn (float a, int b) {
    // nothing inside of {} is visible to linker
    int i = b;      //automatic
    double g;       //automatic
    static double h; //static, not visible to
                    // linker; value retained from call to call
```

body – may access j, f, a, b, i, g, h

```
} //    int fcn( ... )
```

Declaration outside any
function:– *always static*
static storage class:– not
visible to linker



Static Variable Examples (continued)

```
int j;          //static, visible to linker & all functs
static float f; // not visible to linker, visible to
               // to all functions in this program
```

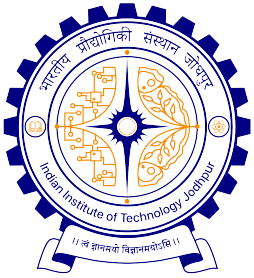
```
int fcn (float a, int b) {
    // nothing inside of {} is visible to linker
```

```
    int i = b;          //automatic
    double g;           //automatic
    static double h; //static, not visible to
                      // linker; value retained from call to call
```

body – may access j, f, a, b, i, g, h

```
} //    int fcn( ... )
```

Inside function:– default is
automatic
static storage class:– not
visible to linker



Static Variable Examples (continued)

```
int j;      //static, visible to linker & all functs
static float f; // not visible to linker, visible to
              // to all functions in this program
```

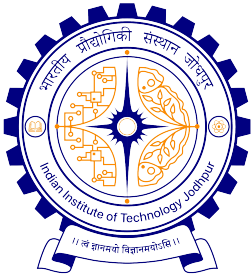
```
int fcn (float a, int b) {
    // nothing inside of {} is visible to linker
```

```
    int i = b;      //automatic
    double g;       //automatic
    static double h; //static, not visible to
                    // linker; value retained from call to call
```

body – may access j, f, a, b, i, g, h

```
} //    int fcn( ... )
```

Value of **h** is also retained across recursions



Extern Variables

```
int j;          //static, visible to linker & all functs
static float f; // not visible to linker, visible to
               // to all functions in this program
```

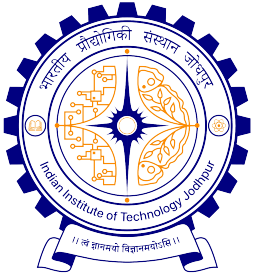
```
extern float p; // static, defined in another program
```

```
int fcn (float a, int b) {
    // nothing inside of {} is visible to linker
    int i = b;    //automatic
    double g;     //automatic
    static double h; //static, not visible to
                  // linker; value retained from call to call
```

body – may access j, f, a, b, i, g, h , p

```
} //    int fcn( ... )
```

extern storage class:– a static variable defined in another C program



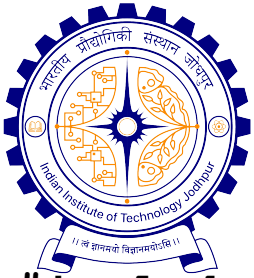
Extern Variables (continued)

- Examples:—

- **stdin**, **stdout**, **stderr** are **extern** variables that point to standard input, output, and error streams.

- **extern** variables:—

- Frequently occur in **.h** files.
- Each must be actually declared outside any function in exactly one **.c** file



Example

A x=0, y=3, g=2
B x=1, y=3, g=4
C x=2, y=3, g=5

```
#include <stdio.h>
```

```
int g = 0; // g and k have storage class "extern"  
extern int k; // g and k are global variables, visible to all functions, exist during whole program execution.
```

```
void fun(char c);
```

```
main() {
```

```
    char c = 'A'; // c and i are storage class "auto".
```

```
    auto int i; // c and i are local to main() only, exist only when main() exists.
```

```
    ++g; fun(c); ++g;
```

```
    fun(c+1); fun(c+2);
```

```
}
```

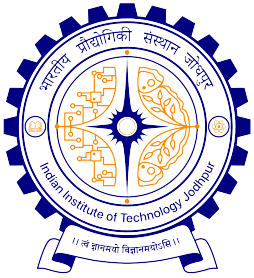
```
void fun(char c) {
```

```
    int y = 3; // y is local, storage class "auto", initialized every time when fun() is evoked, and exists only when fun() is active.
```

```
    static int x=0; // x has storage class "static". x is local to fun(), known only in fun(). x is initialized once, exists during the whole program.
```

```
    putchar(c); printf(" x=%d, y=%d, g=%d\n", x++, y--, ++g);
```

```
}
```



Extern: Multiple Files

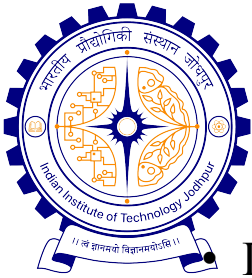
File 1:
main.c

```
#include <stdio.h>
int p(void);
int cnt = 0;
main()
{
    int k;
    cnt = -10;           // Define and initialize cnt as type int, storage class extern.
    k = p();
    printf("%d\n", k);
    ++cnt;
    k = p();
    printf("%d\n", k);
}
```

File 2: p.c

```
extern int cnt;         // Declare cnt as extern. Thus cnt is the same variable defined in File 1.

int p(void)
{
    return ++cnt;
}
```



Type Qualifiers

Declaration of a variable is in the form

[Type qualifiers] [storage class] data type name;

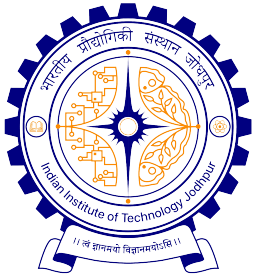
- Examples:

int x; (data type only)

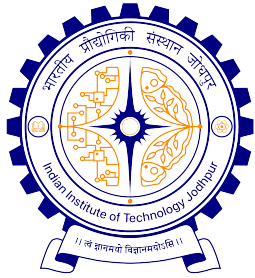
auto int y; (storage class and data type)

const static int z = 101; (type qualifier, storage class, data type)

- **Constant Qualifier:** when a variable or parameter is qualified as const, then the variable or the parameter **cannot occur as a lvalue**, that is, as
 - The left-hand side of an assignment expression.
 - The target of an increment or decrement operation.
- A const variable must be initialized in its definition and the value cannot be changed.
- Example: if we define `const int x = 111`; then the following uses are wrong
 - `x=999; /* ERROR, illegal lvalue */`
 - `++x; /* ERROR illegal lvalue */`



Command Line Arguments

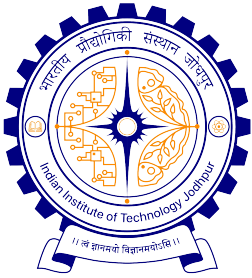


What are Command line arguments?

- In unix we can type, for example,

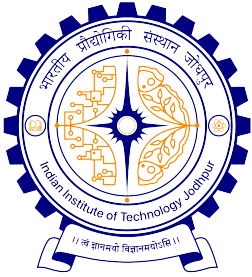
```
$ mv myfile1.dat myfile2.dat
```

- The myfile1.dat and myfile2.dat are known as command line arguments
- But how would we write a C program which could listen to command line arguments?



Passing parameters to main()

- Two parameters will be passed to function main() through command line – argc and argv.
- Name of the parameters are fixed.
- argc is of integer type and it stores the number of parameters (delimited by space) in the command line.
- argv is a 2D array of characters and it stores all the words in the command line.
- By default all the parameters are taken as array of characters (strings) that can be converted to other data types.



Command Line Arguments

```
$ cc -o test test.c
```

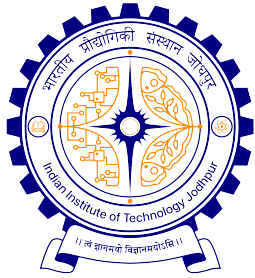
```
$ ./a.out in.dat out.dat
```

```
$ prog_name param_1 param_2 param_3 ..
```

- Command line arguments may be passed by specifying them under `main( )`.

```
int main(int argc, char *argv[ ]);
```

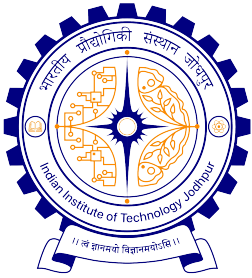
- `argc` tells us the number of command line arguments
- `argv[0]` is the first, `argv[1]` is the second, `argv[2]` is the third etc.



Example code to print arguments

```
int main (int argc, char *argv[]) {  
    int i;  
    for (i= 0; i < argc; i++) {  
        printf ("Argv[%d] = %s", i, argv[i]);  
    }  
    return 0;  
}
```

```
$ a.out how many  
argc = 3  
argv[0] = a.out  
argv[1] = how  
argv[2] = many
```



Passing parameters to main()

```
#include <stdio.h>
```

```
int main(int argc, char *argv[])
```

```
{
```

```
    int i;
```

```
    for(i=0;i<argc;i++) {
```

```
        printf("%d: %s\n",i,argv[i]);
```

```
    }
```

```
    return 0;
```

```
}
```

```
$ ./a.out Tintin 21 8.3
```

```
0: ./a.out
```

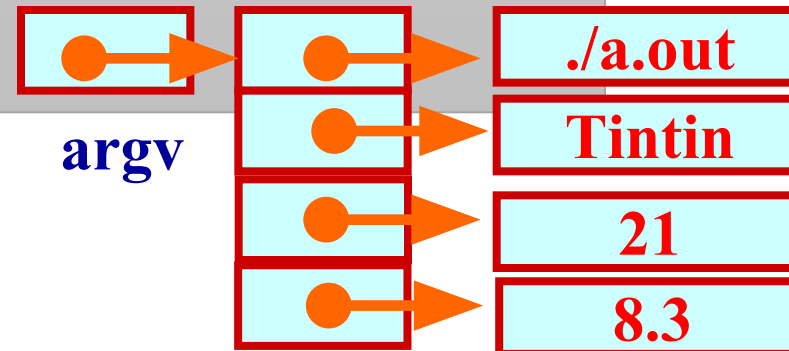
```
1: Tintin
```

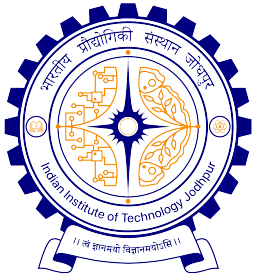
```
2: 21
```

```
3: 8.3
```

```
./a.out Tintin 21 8.3
```

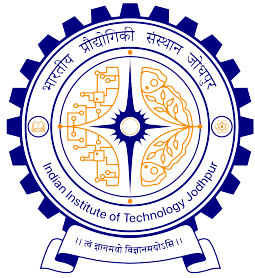
argc=4





All the arguments are passed as strings in `argv[ ]`
But the intention may have been to pass an int/float etc.

Solution: Use `sscanf()`



Library function sscanf()

- Header file:

```
#include <stdio.h>
```

- Function prototype:

```
int sscanf(const char *str, const char *format, ...);
```

- Conversion character is same as scanf().



Passing parameters to main()

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    int i;
    for(i=0;i<argc;i++) {
        printf("%d: %s\n",i, argv[i]);
    }
    return 0;
}
```

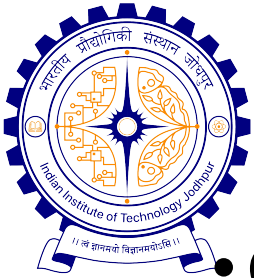
```
$ ./a.out Tintin 21 8.3
0: ./a.out
1: Tintin
2: 21
3: 8.3
$
```

```
#include <stdio.h>
int main(int argc, char *argv[])
{
    char name[20];
    int age;
    float cgpa;

    sscanf(argv[1], "%s", name);
    sscanf(argv[2], "%d", &age);
    sscanf(argv[3], "%f", &cgpa);

    printf("%s %d %f\n",
        name, age, cgpa);
    return 0;
}
```

```
$ ./a.out Tintin 21 8.3
Tintin 21 8.300000
$
```

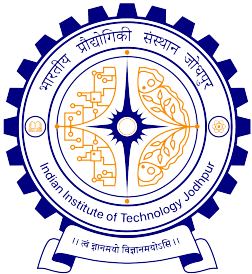



Functions to convert strings to numbers

- Once we've got a string with a number in it (either from a file or from the user typing) we can use `atoi` or `atof` to convert it to a number
- The functions are part of header file `stdlib.h`

```
char numberstring[] = "3.14";  
int i;  
double pi;  
pi = atof (numberstring);  
i = atoi ("12");
```

Both of these functions return 0 if they have a problem



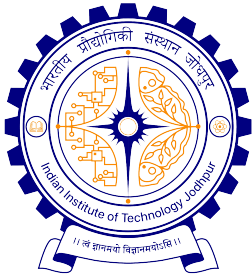
Example: Averaging from Command Line

```
#include <stdio.h>
#include <stdlib.h>
```

```
$ ./a.out 45 239 123
Average=135.666667
```

```
int main(int argc, char *argv[]) {
    float sum=0;
    int i,num;

    num= argc-1;
    for(i=1;i<=num;i++)
        sum+=atof(argv[i]);
    printf("Average=%f \n",sum/(float) num);
}
```



Slide Courtesy: *The C Programming Language*, 2nd edition, by Kernighan and Ritchie and from *C: How to Program*, 5th and 6th editions, by Deitel and Deitel)